Cybersecurity

RESEARCH

Open Access

Improved machine learning-aided linear cryptanalysis: application to DES

Zezhou Hou[1], Jiongjiong Ren[1*] and Shaozhen Chen[1]

Abstract

In CRYPTO 2019, Gohr built a bridge between machine learning and differential cryptanalysis, which show that machine learning-aided methods have advantages over classical differential cryptanalysis. Yet, for linear cryptanalysis, there is lack of effective works showing that machine learning-aided cryptanalysis can reach the benchmark of traditional counterparts and also lack of an effective universal framework using machine learning to assist linear cryptanalysis. In this paper, we mainly focus on machine learning-aided linear cryptanalysis and application to DES. First, we propose a machine learning-aided model to distinguish different Bernoulli distributions and demonstrate the validity of the model through experiments and theoretical analysis. Based on the model, we propose a new machine learning-aided linear cryptanalysis framework, which can be applied to one bit and multiple bits key-recovery attacks. As applications, we perform one bit attacks on 3-, 4-, 5-, 6-round DES and multiple bits attack on 8-round DES. Compared with the previous works about machine learning-aided linear cryptanalysis, the results improve the success rate and the complexity. Most importantly, more rounds are covered in our work. Besides, the work indicates that machine learning-aided cryptanalysis can achieve the same or marginally better performance than classical methods.

Keywords Machine learning, Linear cryptanalysis, Neural distinguisher, Key recovery, DES

Introduction

As a branch of artificial intelligence known as machine learning (ML), it plays the important role and offers a wide range of applications covering many fields, including computer vision, machine reasoning, and natural language processing. And it shares many similarities with cryptography, such as distinguishing, classification, and decision.

As early as 1991, Rivest (1991) explores the relationship between cryptography and machine learning. These years, with the development of hardware and fundamental theory, machine learning has been used in a series of cryptography tasks, including constructing cryptographic schemes (Li et al. 2011; Abadi and Andersen 2016), side-channel cryptanalysis (Maghrebi et al. 2016; Kim et al. 2019), and key agreement (Dong and Huang 2020). The prevailing use of machine learning shows its significant potential for cryptography.

In CRYPTO 2019, there is a significant breakthrough in using machine learning to assist classical cryptanalysis due to Gohr's work (Gohr 2019). Gohr transforms the task of distinguishing ciphertext data from random data into the supervised learning, a branch of machine learning. Quite surprisingly, when applying his framework to SPECK32/64 (Beaulieu et al. 2013), the trained neural network obtains the non-negligible accuracy in classifying random from real ciphertext pairs. His work shows with experiments that the neural networks can be trained to capture the non-randomness that classical differential distinguishers fail to exploit and be substituted for classical differential distinguishers. More importantly, Gohr designs a highly selective key search policy using neural distinguishers ($\mathcal{ND}$s) and Bayesian optimization, which reduces the time complexity by guessing less candidates. Moreover, by adding a classical differential ($\mathcal{CD}$) to the

*Correspondence:
Jiongjiong Ren
jiongjiong_fun@163.com
[1] Information Engineering University, 450000 Zhengzhou, China

beginning of $\mathcal{ND}$s to construct hybrid distinguishers, Gohr performs the key-recovery attack with more rounds.

Inspired by Gohr's work, more and more researchers explore machine learning-aided classical cryptanalysis. In EUROCRYPT 2021, Benamira et al. (2021) concentrate on the inherent workings of Gohr's $\mathcal{ND}$s. They show with experiments that Gohr's $\mathcal{ND}$s generally rely on the differential distribution on the ciphertext pairs, but also on the differential distribution in penultimate and antepenultimate rounds. Their work is helpful to build differential-neural distinguishers for other block ciphers.

In ASIACRYPT 2022, Bao et al. (2022) focus on improving Gohr's key guessing strategy. Bao et al. propose an effective method to enhance the classical component $\mathcal{CD}$ of hybrid distinguishers, whose essence is the neutral bits (**NB**s) (Biham and Chen 2004). By applying various generalized **NB**s in the $\mathcal{CD}$, the improved key-recovery attacks are performed on round-reduced Speck32/64.

There are other works about machine learning-aided cryptanalysis. But most of them mainly focus on using machine learning to assist classical differential cryptanalysis, lack of machine learning-aided other classical cryptanalysis. Especially as a powerful known-plaintext cryptanalysis, linear cryptanalysis has been widely used in various block ciphers. It is immediately meaningful to explore machine learning-aided linear cryptanalysis.

In ESORICS 2020, Hou et al. (2020) propose a method of combining linear cryptanalysis with neural networks. Their work builds the training data by choosing partial bits of plaintext-ciphertext pairs. And the extra padding is performed to make the size of single training sample reach the fixed length. Based on this, Hou et al. perform the one bit key-recovery attacks and the multiple bits key-recovery attacks on round-reduced Des (Standards 1977). However, different from previous works that show the advantage of machine learning over traditional differential cryptanalysis, Hou et al.'s works don't reduce the complexity or increase the success rate, and even perform worse key-recovery attacks than traditional cryptanalysis, which indicates that Hou et al.'s works don't fully utilize neural networks to capture the non-randomness of linear distinguishers. Following Hou et al.'s works, Zhou et al. (2023) design the new neural-linear distinguishers and perform key-recovery attacks for round-reduced Des, which improves upon the results of Hou et al. (2020). However, Zhou et al.'s works don't also reach the baseline of classical techniques[1].

[1] Zhou et al. (2023) use the miscalculated success rate (90% using 4617 plaintexts in the traditional method) to make the comparison with their method. Recalculating the success rate of the traditional method, it can be found that Zhou et al.'s works don't surpass the traditional method. The details are shown in Appendix B.

For this, there is a question to explore as follows:

Can machine learning-aided linear cryptanalysis reach the baseline of traditional counterparts?

Our Contribution. In this paper, we provide an answer to the question by proposing a new machine learning-aided cryptanalysis framework and applying it to linear cryptanalysis. As a basis for the framework, we first introduce a valid model using machine learning to distinguish different Bernoulli distributions. Then we rebuild the training model of machine learning-aided linear cryptanalysis and perform key-recovery attacks, whose success rate is closed to or marginally higher than that of traditional linear cryptanalysis with using the same (or less) number of plaintexts. These results are summarized in Table 1. Our work provides strong positive examples and extensive experimental data to support that ML-aided methods also have advantages for linear cryptanalysis. The details of our contribution include the following.

- **Model of distinguishing different Bernoulli distributions using neural networks.** An applicable data format for training neural networks is devised. We demonstrate the validity through various experiments and theoretical analysis, which shows that it is feasible to distinguish different Bernoulli distributions using the given data format and neural networks. And additional experiments provide rules of thumb for improving the accuracy of distinguishing different Bernoulli distributions, which provide the basis of building neural distinguishers for linear cryptanalysis.
- **Framework of machine learning-aided one bit key-recovery attack.** For one bit key-recovery attack in linear cryptanalysis, we transform the task of recovering one bit into the task of distinguishing different Bernoulli distributions. Based on this, we design a training data format and propose a new framework for one bit key-recovery attack, which is applied to round-reduced Des, whose success rate is close to that of classical attacks. And the results show that our work significantly surpasses Hou et al.'s work (Hou et al. 2020) and Zhou et al.'s work (Zhou et al. 2023) in the number of rounds and success rates.
- **Framework of machine learning-aided multiple bits key-recovery attack.** For multiple bits key-recovery attack in linear cryptanalysis, we design another training data format used to classify random data from ciphertext data and apply it to the key-recovery attack on 8-round Des. For comparison, a classical attack is performed with the same number of plaintexts, which shows that machine learning-aided linear cryptanalysis can achieve

Table 1 Summary of linear cryptanalysis on DES

Target	Rounds	Data	Success Rate	Type of Attack
DES (One Bit)	3	26	97.99%	Classical (Matsui 1993)
	3	32	95%	ML-aided (Hou et al. 2020)
	3	32	98.5%	ML-aided (Zhou et al. 2023)
	3	26	98.06%	ML-aided
	3	32	98.96%	ML-aided
	4	269	97.89%	Classical (Matsui 1993)
	4	332	95%	ML-aided (Hou et al. 2020)
	4	256	96.6%	ML-aided (Zhou et al. 2023)
	4	252	97.61%	ML-aided
	4	264	97.79%	ML-aided
	5	2751	97.68%	Classical (Matsui 1993)
	5	8631	90%	ML-aided (Hou et al. 2020)
	5	1024	84.28%	ML-aided (Zhou et al. 2023)
	5	1008	88.39%	ML-aided
	5	2660	97.64%	ML-aided
	6	68939	97.63%	Classical (Matsui 1993)
	6	2048	60.6%	ML-aided (Zhou et al. 2023)
	6	2016	63.64%	ML-aided
	6	68928	97.77%	ML-aided
DES (Multiple Bits)	4	-	-	ML-aided (Hou et al. 2020)
	5	-	-	ML-aided (Zhou et al. 2023)
	5	240	25.6%	ML-aided
	8	4×10^5	78.6%	Classical (Matsui 1993)
	8	4×10^5	80.2%	ML-aided

the same performance as traditional counterparts. In addition, compared with Hou et al.'s work (Hou et al. 2020) and Zhou et al.'s work (Zhou et al. 2023), our method has considerable advantages in success rate and the number of rounds.

ORGANIZATION. The rest of the paper is organized as follows. Section 2 introduces notations as well as basic descriptions that will be used in the rest of the paper. Section 3 introduces the method distinguishing different Bernoulli distributions using neural networks. Section 4 proposes machine learning-aided linear cryptanalysis and performs key-recovery attacks on round-reduced DES. Finally, we study possible improvements in Sect. 5.

Preliminaries

Notations

Denote by $2n$ the state size in bits. For DES, n is 32. Denote by (L_i, R_i) the left and right branches of a state after the encryption of i rounds. Denote by $L_i||R_i$ the concatenation of L_i and R_i. Denote by $L_i[j]$ (resp. $R_i[j]$) the j-th bit of L_i (resp. R_i) counted starting from 0, where $L_i[0]$ (resp. $R_i[0]$) is the LSB (Least Significant Bit) of L_i (resp. R_i), i.e. $L_i = L_i[n-1]||L_i[n-2]||...||L_i[0]$ and $R_i = R_i[n-1]||R_i[n-2]||...||R_i[0]$. Denote by $L_i[i_1, i_2, .., i_t]$ the set of i_1-th bit,...,i_t-th bit of L_i. Denote by $|x|$ the absolute value of x. Denote by $\oplus$ the bit-wise XOR.

Brief description of DES

DES is an iterative cryptographic algorithm with Feistel structure. The encryption algorithm of DES is listed in Algorithm 1. A 64-bit plaintext block P is subject to an initial permutation IP to form L_0 and R_0 (32 bits each). Its non-linear function $F(.)$ contains four operations which include bit selection, XOR, S-box operation and permutation operation. Then a 64-bit ciphertext block C is generated subject to a final permutation IP^{-1}. Its 48-bit subkey K_i is generated by the 56-bit master key following its key schedule function. More details can be seen in Standards (1977). In this paper, we omit the initial permutation IP and the final permutation IP^{-1}.

Algorithm 1 Encryption of DES

Require: P, $\{K_0, K_1, ..., K_{15}\}$
Ensure: C
1: $(L_0, R_0) \leftarrow IP(P)$
2: **for** $i = 0$ to 15 **do**
3: $\quad L_{i+1} \leftarrow R_i$
4: $\quad R_{i+1} \leftarrow L_i \oplus F(R_i, K_i)$
5: **end for**
6: $C \leftarrow IP^{-1}(R_{16}||L_{16})$
7: return C

Matsui's algorithms

Linear cryptanalysis is a known-plaintext attack proposed by Matsui (1993, 1994). In Matsui (1993), Matsui presents a statistical method to break DES using linear approximate expressions. Given the linear approximate expressions of r-round target cipher (see Eq. 1), the expression holds with probability $p_r \neq 1/2$ for the randomly given plaintext P, the master key K and the corresponding ciphertext C, where α, β and γ are the bit location masks.

$$\alpha \cdot P \oplus \beta \cdot C = \gamma \cdot K \tag{1}$$

Given N plaintexts and their corresponding ciphertexts using the same master key K, Matsui gives an effective algorithm based on the maximum likelihood method to determine the value of $\gamma \cdot K$ as follows (**Matsui's algorithm 1**) (Matsui 1993):

Step 1. Let T the number of plaintexts that makes $\alpha \cdot P \oplus \beta \cdot C$ be equal to 0.

Step 2. If $(T > \frac{N}{2}$ and $p_r > \frac{1}{2})$ or $(T < \frac{N}{2}$ and $p_r < \frac{1}{2})$, then guess $\gamma \cdot K = 0$, else guess $\gamma \cdot K = 1$.

Moreover, Matsui applies the r-round linear expression to deduce multiple key bits. As shown in Fig. 1, if the guess of K_0 and K_{r+1} is incorrect, then $\alpha \cdot P' \oplus \beta \cdot C' = \gamma \cdot K'$ holds with the probability closed to $\frac{1}{2}$. Given N plaintexts and their corresponding ciphertexts using the same master key K, the maximum likelihood method applied to deduce K_0, K_{r+1} and $\gamma \cdot K'$ is as follows (**Matsui's algorithm 2**) (Matsui 1993):

Step 1. For each candidate GK_i of $K_0||K_{r+1}$, let T_i the number of plaintexts that makes $\alpha \cdot P' \oplus \beta \cdot C'$ be equal to 0.

Step 2. Let T_{max} be the maximal value and T_{min} be the minimal values.

Step 3. If $|T_{max} - \frac{N}{2}| > |T_{min} - \frac{N}{2}|$, then the value of $K_0||K_{r+1}$ is the candidate corresponding T_{max} and guess $\gamma \cdot K' = 0$ (when $p_r > \frac{1}{2}$) or 1 (when $p_r < \frac{1}{2}$).

Step 4. If $|T_{max} - \frac{N}{2}| < |T_{min} - \frac{N}{2}|$, then the value of $K_0||K_{r+1}$ is the candidate corresponding T_{min} and guess $\gamma \cdot K' = 0$ (when $p_r < \frac{1}{2}$) or 1 (when $p_r > \frac{1}{2}$).

Following Matsui's work, more researchers try to utilize different approximate expressions to improve key-recovery attacks (Jr. and Robshaw 1994; Langford and Hellman 1994; Harpes et al. 1995; Knudsen and Robshaw 1996; Courtois 2004; Hermelin et al. 2008). Even if there are various extensions of linear cryptanalysis, their core in the key-guessing phase is the maximum likelihood method proposed by Matsui.

Neural distinguishers and combined response distinguishers

In Gohr (2019), Gohr gives an effective method training neural networks to capture the non-randomness of target block ciphers. Given a particular difference Δ_{in} and a target cipher, the neural network is trained to distinguish ciphertext pairs between using Δ_{in} as the input difference and using random differences as the input differences.

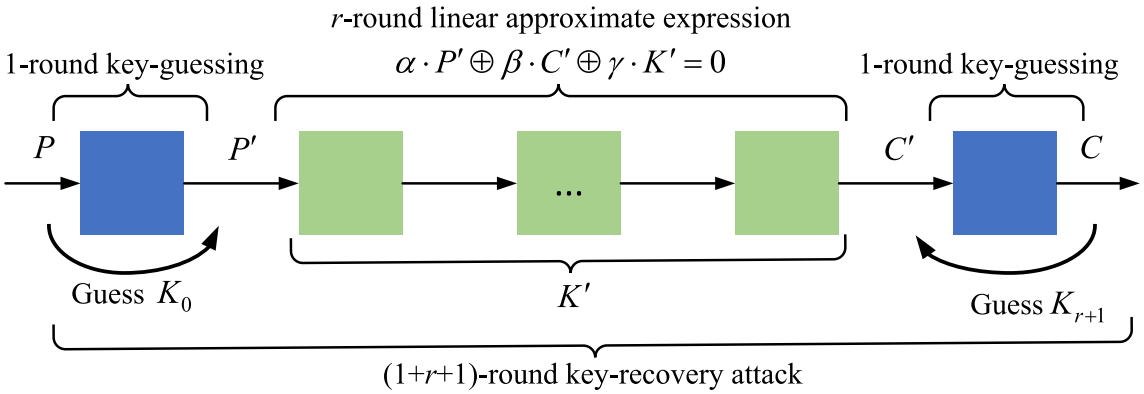


Fig. 1 Components of the multiple bits key-recovery attack

These ciphertext pairs using Δ_{in} as the input difference are labeled by 1. And the ciphertext pairs using random differences as the input differences are labeled by 0.

Given enough training samples, Gohr trains a deep residual network (denoted by ResNets, He et al. 2016) to distinguish between ciphertext data and random data. The trained neural network plays the role of distinguisher in cryptanalysis. Feeding a pair of ciphertexts into the trained neural network, the trained neural network returns a value v ranging from 0 to 1. If $v > 0.5$, then the pair of ciphertexts are considered to use Δ_{in} as the input difference, else the pair of ciphertexts are considered to use an random difference as the input difference. The trained neural network is called neural distinguishers ($\mathcal{ND}$s). As a new technique different from previous distinguishers, there are some limitations in using $\mathcal{ND}$s. When the accuracy becomes marginally higher than 0.5, it is hard to be used in the key-recovery attacks.

$$v_{crd} = \sum_{i=1}^{t} \log_2 \left(\frac{v_i}{1 - v_i} \right) \tag{2}$$

Thus, Gohr uses the combined response (see Eq. 2) of the $\mathcal{ND}$ over large number of samples of the same distribution as a distinguisher (named as combined-response-distinguisher, $\mathcal{CRD}$). More details of $\mathcal{CRD}$ can be found in Bao et al. (2021).

Machine learning and bernoulli distribution

This section shows that the neural networks can be used to distinguish between different Bernoulli distributions. We design an effective model to help neural networks to capture the differences between different Bernoulli distributions. Further, by virtue of the connection between Bernoulli distribution and approximate expressions, we propose a method to capture the feature of approximate expressions using neural networks.

Distinguish different bernoulli distributions

Before exploring how to distinguish different Bernoulli distributions using neural networks, we first describe the problem as follows.

Problem 1 Let $Bern(p)$ and $Bern(q)$ be two different Bernoulli distributions, where $0.25 < q < p < 0.75$. Let $S := \{x_1, x_2, ..., x_l\}$ be a set of random variables, where all random variables obey the same Bernoulli distribution $Bern(p)$ or $Bern(q)$. So how to use a neural network to determine whether these random variables are from $Bern(p)$ or $Bern(q)$?

Before solving the Problem 1, we first consider two sub-problems, as follows:

P1. How to construct the training data set and verification data set.

P2. How to build the neural network.

For *P1*, we design an appropriate data format. The $\overline{x_1}||\overline{x_2}||...||\overline{x_t}$ is regarded as a sample, where all x_i follow the same distribution and $\overline{x_i}$ is the value of x_i. The length of a sample t is called the sample's dimension ($\mathcal{DIM}$). Each sample will be attached with a label Y:

$$Y(\overline{x_1}||\overline{x_2}||...||\overline{x_t}) = \begin{cases} 1 \ if \ x_i \sim Bern(p), i \in [1,t] \\ 0 \ if \ x_i \sim Bern(q), i \in [1,t] \end{cases}. \tag{3}$$

In Eq. 3, the concatenation of the values obeying $Bern(p)$ is labeled by 1 (denoted as $Bern(p) \to 1$), and the concatenation of the values obeying $Bern(q)$ is labeled by 0 (denoted as $Bern(q) \to 0$). And consistent with the terminology used in deep learning, these samples labeled by 1 are called positive examples and these samples labeled by 0 are called negative examples.

For *P2*, we choose the neural networks ResNets used in Gohr (2019), and only the input layer is modified to match the shape of training data.

In order to illustrate the validity of the data format and the neural network, we do the following experiment (**Experiment A**):

Step 1. Obtain $\left\{\overline{x_1}, \overline{x_2}, ..., \overline{x_{\frac{N}{2}\times t}}, \overline{y_1}, \overline{y_2}, ..., \overline{y_{\frac{N}{2}\times t}}\right\}$, where $x_i \sim Bern(p)$ and $y_i \sim Bern(q), i \in \left[1, \frac{N}{2} \times t\right]$.

Step 2. Generate training data set $TS := \{\overline{x_{i\times t+1}}||\overline{x_{i\times t+2}}||...||\overline{x_{i\times t+t}}\} \cup \{\overline{y_{i\times t+1}}||\overline{y_{i\times t+2}}||...||\overline{y_{i\times t+t}}\} (i \in \left[0, \frac{N}{2} - 1\right])$. Each of samples in TS is attached with a label following Eq. 3.

Step 3. Similar to the generation of TS, generate N' samples for verification data set VS.

Step 4. Train the neural network using TS and VS.

Step 5. Similar to the generation of TS, generate N'' samples for the testing data. Among the testing data, half are labeled by 1 and half are labeled by 0. Use the trained neural network to predict the labels of testing data. Calculate the accuracy of the trained neural network.

In **Experiment A**, $p = 0.5 + 10^{-2}$, $q = 0.5 - 10^{-2}$, $t = 64$, $N = 2^{24}$, $N' = 2^{21}$, $N'' = 2^{21}$. With these parameters, the accuracy of the trained neural network is about 56.3%. The trained neural network can be used to distinguish $Bern(0.5 + 10^{-2})$ and $Bern(0.5 - 10^{-2})$.

As shown in **Experiment A**, the data format and the trained neural network are effective. But in **Experiment A**, the values of p, q and t are limited to a fixed value, which may be not general for other values of p and q.

For the same neural network structure, with that only the input layer is modified to fit $\mathcal{DIM}$, we believe that the accuracy is only related to $|p-q|$ and $\mathcal{DIM}$. With that, we propose three conjectures.

Conjecture 1 *Let Acc be the accuracy of the trained neural network for Bern*(p) *and Bern*(q)*. Let Acc*$'$ *be the accuracy of another trained neural network for Bern*(p') *and Bern*(q')*. Given a fixed* $\mathcal{DIM}$ *and the same neural network structure,* $Acc \approx Acc'$ *with* $|p-q| = |p'-q'|$*, where* $0.25 < p, q, p', q' < 0.75$*.*

To verify Conjecture 1, we perform the following experiment (**Experiment B**):

Step 1. Randomly generate q ranging from 0.25 to $0.75-\delta$.

Step 2. Obtain the accuracy *Acc*, where the sample from *Bern*(q) is labeled by 0 and the sample from *Bern*$(q+\delta)$ is labeled by 1.

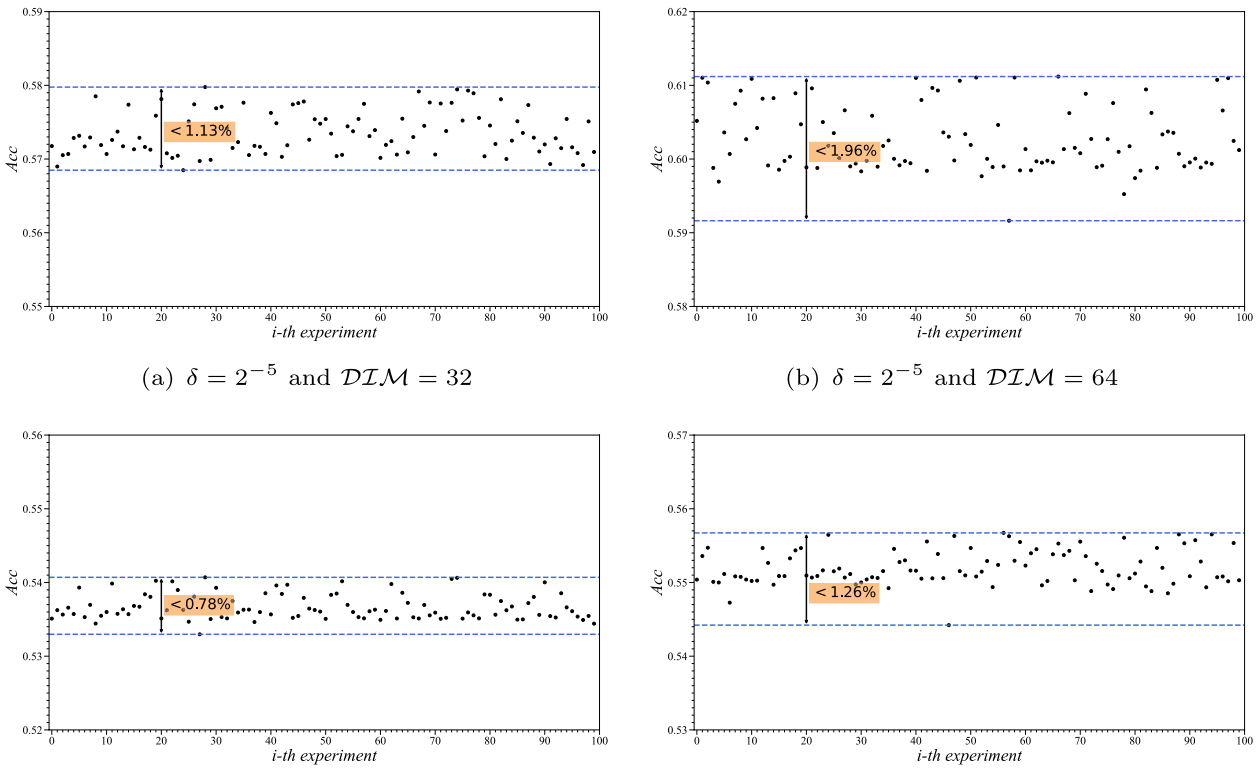
Perform **Experiment B** 100 times with $\delta = 2^{-5}$ and $\mathcal{DIM} = 32$ (and $\mathcal{DIM} = 64$). Another 100 trials are performed using $\delta = 2^{-7}$ and $\mathcal{DIM} = 128$ (and $\mathcal{DIM} = 256$). The result is shown in Fig. 2.

This reveals that the accuracy fluctuates with a small rhythm. For 100 randomly generated tuple (p, q), the accuracy of neural networks fluctuates within a small range given the fixed value of $|p-q|$ and $\mathcal{DIM}$. That is to say, the specific value of p and q has a negligible influence on the accuracy of distinguishing between *Bern*(q) and *Bern*(p). And we only need to take the influence of the value of $|p-q|$ on the accuracy into our account, not considering the influence of the specific value of p and q.

Conjecture 2 *Given a fixed* $\mathcal{DIM}$ *and the same neural network structure, the accuracy of the trained neural network Acc will increase if the value of* $|p-q|$ *increases, where* $0.25 < p, q < 0.75$*.*

Conjecture 3 *Given the fixed values of p, q and the same neural network structure, the accuracy of the trained neural network Acc will increase if the* $\mathcal{DIM}$ *increases, where* $0.25 < p, q < 0.75$*.*

In support of Conjecture 2 and Conjecture 3, we rerun **Experiment B** using other δ and $\mathcal{DIM}$, which are shown in Table 2 and Fig. 3. Given the fixed $\mathcal{DIM}$, the accuracy increases with increasing the value of δ, where $\delta = |p-q|$. Similarly, given the fixed δ, the accuracy increases with increasing the length of single sample $\mathcal{DIM}$.



(a) $\delta = 2^{-5}$ and $\mathcal{DIM} = 32$

(b) $\delta = 2^{-5}$ and $\mathcal{DIM} = 64$

(c) $\delta = 2^{-7}$ and $\mathcal{DIM} = 128$

(d) $\delta = 2^{-7}$ and $\mathcal{DIM} = 256$

Fig. 2 Fluctuation range of *Acc*

Table 2 *Acc* of using multiple δ and $\mathcal{DIM}$

δ	$\mathcal{DIM}$								
	32	**48**	**64**	**80**	**96**	**112**	**128**	**144**	**160**
2^{-4}	64.52%	66.90%	70.49%	71.35%	73.12%	74.99%	75.96%	77.60%	78.66%
2^{-5}	57.42%	58.59%	60.62%	62.06%	62.02%	63.34%	64.11%	64.70%	67.03%
2^{-6}	53.92%	54.63%	55.09%	55.54%	56.09%	57.01%	57.42%	58.08%	58.89%

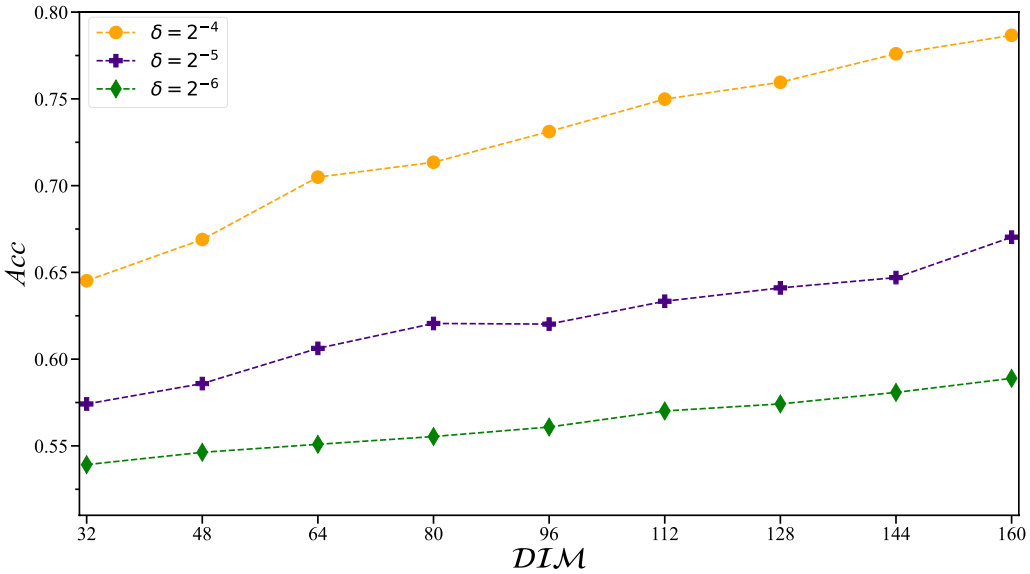


Fig. 3 *Acc* of using multiple δ and $\mathcal{DIM}$

Moreover, Conjecture 2 and Conjecture 3 describe an effective method to improve the accuracy of distinguishing between $Bern(p)$ and $Bern(q)$, that is increasing the length of single sample $\mathcal{DIM}$ for the fixed p and q.

Let's review Problem 1, it can be solved with the given training data format and three conjectures, as follow:

Step 1. Let l be the $\mathcal{DIM}$, and train neural networks denoted by NN_l.

Step 2. If the NN_l is noneffective (the accuracy is lower than 51%), then reperform **Step 1** using other neural network, else perform **Step 3**.

Step 3. Feed the values of these variables to NN_l and obtain the output of NN_l (denoted by v).

Step 4. Estimate the distribution of these variables using v. If $v > 0.5$, then guess these variables obey $Bern(p)$, else guess these variables obey $Bern(q)$.

Improve the accuracy using $\mathcal{CRD}$

In Gohr (2019); Bao et al. (2022), the combined-response-distinguishers ($\mathcal{CRD}s$) are used to improve the accuracy distinguishing between positive examples and negative examples. Considering that the $\mathcal{CRD}$ only uses Eq. 2 to combine response of the neural networks over large number of samples of the same distribution, it is easy to apply the method of $\mathcal{CRD}$ to the trained neural networks of distinguishing between $Bern(p)$ and $Bern(q)$. The $\mathcal{CRD}$ of distinguishing between $Bern(p)$ and $Bern(q)$ can be constructed as follows:

Step 1. Train an effective neural network denoted by NN_n, where n is the sample's dimension.

Step 2. Let $S := \{x_1, x_2, ..., x_{n\times t}\}$ be $n \times t$ random variables obeying the same Bernoulli distribution $Bern(p)$ or $Bern(q)$.

Step 3. Concatenate these values of random variables to obtain t samples $S_S := \{\overline{x_1}||\overline{x_2}||...||\overline{x_n}, ..., \overline{x_{n\times(t-1)+1}}||\overline{x_{n\times(t-1)+2}}||...||\overline{x_{n\times t}}\}$.

Step 4. Feed these samples to NN_n and obtain the output $v_0, v_1, ..., v_{t-1}$, where v_i is the output using $\overline{x_{in+1}}||...||\overline{x_{in+n}}$ as the input of NN_n.

Step 5. Estimate the distribution of these variables using $\sum_{i=0}^{t-1} \log_2\left(\frac{v_i}{1-v_i}\right)$. If $\sum_{i=0}^{t-1} \log_2\left(\frac{v_i}{1-v_i}\right) > 0$, then guess these variables obey $Bern(p)$, else guess these variables obey $Bern(q)$.

Example 1 Given $p = 0.5 + 2^{-5}$ and $q = 0.5 - 2^{-5}$, train the neural network with $\mathcal{DIM} = 32$, denoted by NN_{32}. The accuracy of NN_{32} is about 63.67%. And the accuracy of $\mathcal{CRD}$s, with performing 2^{18} experiments, is seen in Table 3 when combining $2^0 \sim 2^8$ samples obeying the same distribution, which shows that the method of $\mathcal{CRD}$ is effective for improving the results of distinguishing between $Bern(p)$ and $Bern(q)$.

In $\mathcal{CRD}$, two parameters are important for the distinguishing effect, that is the length of single sample $\mathcal{DIM}$ and the number of samples (denoted by $\mathcal{NS}$). The ordered tuple $(\mathcal{DIM}, \mathcal{NS})$ can be regarded as the parameter of $\mathcal{CRD}$, where the $\mathcal{CRD}$ can be denoted as $\mathcal{CRD}^{\mathcal{NS}}_{\mathcal{DIM}}$. And for $\mathcal{CRD}^{\mathcal{NS}}_{\mathcal{DIM}}$, $\mathcal{DIM} \times \mathcal{NS}$ random variables are used in the sample set.

In Conjecture 3, it is verified that increasing the length of single sample can also improve the results of distinguishing between $Bern(p)$ and $Bern(q)$. So what should be regarded as the first thing to improve the distinguishing effect? The length of the single sample or the number of samples of $\mathcal{CRD}$s? In the rest of this section, we are exploring the following practical question.

Problem 2 Let $S := \{x_1, x_2, ..., x_{\alpha\times\beta\times t}\}$ be a set of random variables, where all random variables obey the same Bernoulli distribution $Bern(p)$ or $Bern(q)$. Let NN_α and NN_β be two effective trained neural networks, where α (resp.β) is the length of the single sample for NN_α (resp.NN_β) and $\alpha < \beta$. These values of random variables can be incorporated into $\beta \times t$ samples for NN_α or $\alpha \times t$ samples for NN_β. So which ordered tuple should be chosen, (α, β) or (β, α)?

To challenge Problem 2, we test multiple ordered tuples with multiple p and q, which is shown in Fig. 4. The experimental results show that the accuracy of $\mathcal{CRD}^\beta_\alpha$ is different from the accuracy of $\mathcal{CRD}^\alpha_\beta$ with that the same number of variables is used. Take $p = 0.5 + 2^{-5}$ and $q = 0.5 - 2^{-5}$ for example, using 1280 variables, the accuracy with $\mathcal{DIM} = 160$ is higher than the the accuracy with $\mathcal{DIM} = 32$, which shows that the length of single sample is more important. But for $p = 0.5 + 2^{-6}$ and $q = 0.5 - 2^{-6}$, using 1280 variables, the accuracy with $\mathcal{DIM} = 32$ is higher than the accuracy with $\mathcal{DIM} = 128$, which shows that the number of samples is more important. The results indicate that it is not sure that the $\mathcal{CRD}^\beta_\alpha$ has the better distinguishing effect than the $\mathcal{CRD}^\alpha_\beta$ with $\alpha < \beta$. It is difficult to decide which ordered tuple should be used for (α, β) and (β, α). In the specific work, there require a great deal of experimentation to choose the appropriate length of single sample and the appropriate number of samples.

Bernoulli distribution and linear approximation

In Matsui (1993, 1994), Matsui uses linear approximate expressions (Eq. 1) to attack target ciphers. Considering the variant of Eq. 1:

$$\alpha \cdot P \oplus \beta \cdot C \oplus \gamma \cdot K = 0, \tag{4}$$

Eq. 4 holds with probability $p_r \neq 1/2$ for the randomly given plaintext P, the master key K and the corresponding ciphertext C.

And the $\alpha \cdot P \oplus \beta \cdot C \oplus \gamma \cdot K$ can be regard as a random variable obeying Bernoulli distribution $Bern(1 - p_r)$. As a variant of Eq. 3, another data format is shown in Eq. 5. In Eq. 5, for the positive examples, t ordered triples (P_i, C_i, K_i) are transformed to t bits, which are cascaded into a sample. In Eq. 5, $i \in [1, t]$.

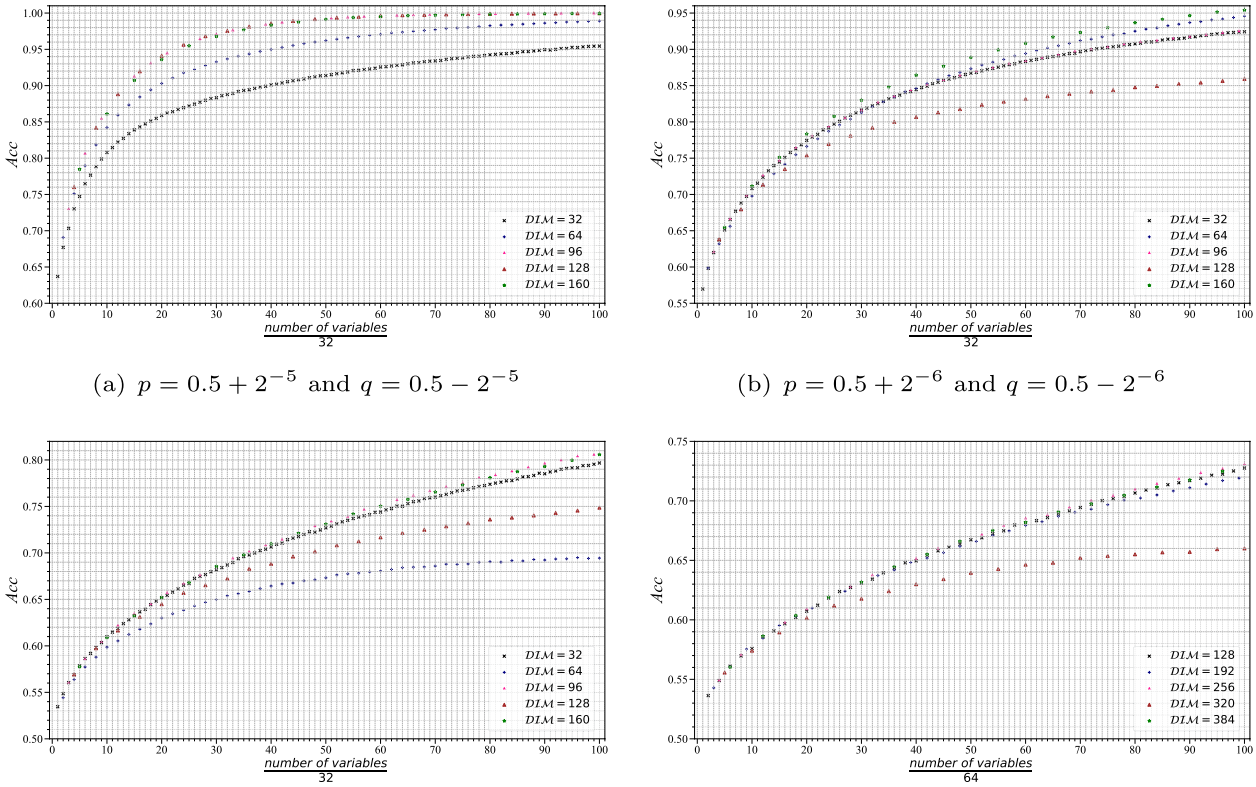
$$Y(x_1||x_2||...||x_t) = \begin{cases} 1 \ if \ x_i = \alpha \cdot P_i \oplus \beta \cdot C_i \oplus \gamma \cdot K_i \\ 0 \ if \ x_i \sim Bern\left(\frac{1}{2}\right) \end{cases} \tag{5}$$

Example 2 In Matsui (1993), Matsui proposes the 3-round linear expression of DES (see L_3 in Table 4), which holds with probability $p_r \approx 1/2 + 1.56 \times 2^{-3}$. Using the given data format, we train the neural networks to distinguish between cipher data $(Bern(1 - p_r))$ and random data $(Bern\left(\frac{1}{2}\right))$. Given 2^{24} training samples and 2^{21} verification samples, the accuracy of the trained neural network is about 100% for $\mathcal{DIM} = 32$.

Similar to L_3 in Table 4, given $L_0[7, 18, 24, 29]$, $R_0[15]$, $L_3[7, 18, 24, 29]$, $R_3[15]$, $K_0[22]$ and $K_2[22]$, the trained neural networks of Example 2 could be used to distinguish between ciphertext data of 3-round DES and random data. In other words, the trained neural networks could replace L_3 in Table 4 as a distinguisher of 3-round DES. Further,

Table 3 *Acc* of $\mathcal{CRD}$s with $p = 0.5 + 2^{-5}, q = 0.5 - 2^{-5}$ and $\mathcal{DIM} = 32$

number of samples	2^0	2^1	2^2	2^3	2^4	2^5	2^6	2^7	2^8
Acc	63.66%	67.71%	73.02%	78.89%	84.28%	88.74%	92.91%	96.74%	99.18%



(a) $p = 0.5 + 2^{-5}$ and $q = 0.5 - 2^{-5}$

(b) $p = 0.5 + 2^{-6}$ and $q = 0.5 - 2^{-6}$

(c) $p = 0.5 + 2^{-7}$ and $q = 0.5 - 2^{-7}$

(d) $p = 0.5 + 2^{-8}$ and $q = 0.5 - 2^{-8}$

Fig. 4 The accuracy curve when combining a various number of samples from the same distribution

Table 4 Linear approximate expressions of 3-, 4-, 5-, 6-round DES (Matsui 1994)

L_i	linear approximate expressions
L_3	$(\oplus_{i\in[7,18,24,29]} L_0[i]) \oplus R_0[15] \oplus (\oplus_{i\in[7,18,24,29]} L_3[i]) \oplus R_3[15] = K_0[22] \oplus K_2[22]$
L_4	$(\oplus_{i\in[7,18,24,29]} L_0[i]) \oplus R_0[15] \oplus (\oplus_{i\in[15]} L_4[i]) \oplus (\oplus_{i\in[7,18,24,27,28,29,30,31]} R_4[i])$ $= K_0[22] \oplus K_2[22] \oplus (\oplus_{i\in[42,43,45,46]} K_3[i])$
L_5	$L_0[15] \oplus (\oplus_{i\in[7,18,24,27,28,29,30,31]} R_0[i]) \oplus L_5[15] \oplus (\oplus_{i\in[7,18,24,27,28,29,30,31]} R_5[i])$ $= (\oplus_{i\in[42,43,45,46]} K_0[i]) \oplus K_1[22] \oplus K_3[22] \oplus (\oplus_{i\in[42,43,45,46]} K_4[i])$
L_6	$(\oplus_{i\in[7,18,24]} L_0[i]) \oplus (\oplus_{i\in[7,18,24,29]} L_6[i]) \oplus R_6[15] = K_1[22] \oplus K_2[44] \oplus K_3[22] \oplus K_5[22]$

Example 2 indicates that it is possible replacing linear approximation with an effective neural network.

ML-aided linear cryptanalysis

This section shows that the neural networks could be harnessed to cooperate with linear approximations and perform key-recovery attacks competitive to the attacks devised by Matsui's algorithms. In the following, similar to one bit key recovery and multiple bits key recovery of orthodox linear attacks, we present two linear attack architectures employing neural networks, which are used in one bit attack and multiple bits attack. Comparisons between machine learning-aided linear cryptanalysis and previous works are made accordingly. The results show that the new architectures can achieve the same performance as orthodox cryptanalysis.

ML-aided one bit key recovery

Attack architecture

Reviewing Eq. 1, for the fixed master key K, the randomly given plaintext P and the corresponding ciphertext C, the $\alpha \cdot P \oplus \beta \cdot C$ obeys the Bernoulli distribution and the parameter of the distribution is related to $\gamma \cdot K$. If $\gamma \cdot K = 1$, $\alpha \cdot P \oplus \beta \cdot C \sim Bern(p_r)$, otherwise $\alpha \cdot P \oplus \beta \cdot C \sim Bern(1 - p_r)$. Considering that $\alpha \cdot P \oplus \beta \cdot C$ obeys the different distribution related to

$\gamma \cdot K$, it is possible to train neural networks and perform key-recovery attacks.

Generation of the training data. For a target cipher and its r-round linear approximate expression (Eq. 1), the neural network is trained to distinguish the different distributions of $\alpha \cdot P \oplus \beta \cdot C$, where these distributions are related to the value of $\gamma \cdot K$. And each of the training data is a multi-dimensional vector together with a label taking a value 0 or 1, which is shown in Eq. 6. In Eq. 6, $\omega_i = \alpha \cdot P_i \oplus \beta \cdot C_i \oplus \gamma \cdot K, \ \ i \in [1, t]$. For the multi-dimensional vector $x = x_1||...||x_t$, its single element x_i is generated by P_i, K and the corresponding C_i. And in the training data set, one sample only uses one master key K and different samples use different master keys. For N training samples, it needs $N \times t$ plaintexts and N master keys following the uniform distribution.

$$Y(x_1||x_2||...||x_t) = \begin{cases} 1 \ \ if \ \ x_i = \omega_i \ \ (x_i \sim Bern(1-p_r)) \\ 0 \ \ if \ \ x_i = \omega_i \oplus 1 \ \ (x_i \sim Bern(p_r)) \end{cases} \tag{6}$$

Training schemes. The neural networks used in key-recovery attacks are the same as the neural networks used in Gohr (2019), where only the input layer is modified to match the shape of training data. And the reason for choosing the neural networks used in Gohr (2019) is shown in Appendix A. For the r-round linear approximate expression, the trained neural network is denoted as ${ND_r}^t$ if its accuracy is higher than 51%, where t is the number of plaintexts used in one sample.

Key-recovery attack. Given the $\gamma \cdot K$ and the corresponding plaintext-ciphertext pairs $\{(P_1, C_1), ..., (P_N, C_N)\}$, the $\alpha \cdot P_i \oplus \beta \cdot C_i$ obeys the Bernoulli distribution $Bern(p_r)$ or $Bern(1-p_r)$. And if $\gamma \cdot K = 1$, $\alpha \cdot P_i \oplus \beta \cdot C_i \sim Bern(p_r)$, otherwise $\alpha \cdot P_i \oplus \beta \cdot C_i \sim Bern(1-p_r)$. In other words, if $\alpha \cdot P_i \oplus \beta \cdot C_i \sim Bern(p_r)$, we can guess that the value of $\gamma \cdot K$ is 1, otherwise the value of $\gamma \cdot K$ is 0. The details of ML-aided one bit key recovery attack are as follows:

Step 1. Train the neural network and the length of a single sample is t. The trained neural network is denoted as ${ND_r}^t$, where r is the number of rounds.

Step 2. Given $N \times t$ plaintext pairs and their corresponding ciphertext pairs, transform these plaintext-ciphertext pairs into N samples by $\alpha \cdot P \oplus \beta \cdot C$.

Step 3. Feed these samples into the trained neural network and obtain the return values v_i of ${ND_r}^t$, where v_i is the return value of the i-th sample.

Step 4. If $\sum_{i=1}^{N} \log_2 \left(\frac{v_i}{1-v_i} \right) / N > 0$, then guess $\gamma \cdot K = 0$, else guess $\gamma \cdot K = 1$.

In the key-recovery algorithm, there are three main parameters need to be fixed, which are the linear approximate expression L_r, the length of single training sample t and the size of plaintext-ciphertext pairs set $N \times t$, respectively. These three parameters are related to the success rate.

Application to DES

Before performing the key-recovery attacks of round-reduced DES, the linear approximate expressions are used to train neural networks. The linear approximate expressions of 3-, 4-, 5-, 6-round DES are shown in Table 4.

For the linear approximate expression L_i, we train multiple neural networks and perform key-recovery attacks with $N \times t = \left({p_r}^i - 0.5\right)^{-2}$, where ${p_r}^i$ is the probability of L_i. The results are shown in Table 5.

Data Complexity. There are $N \times t$ plaintexts used in key-recovery attacks.

Table 5 Comparison with Hou et al. (2020) and Matsui's Algorithm 1

Target		ML-aided one bit key-recovery attack			Hou *et al.*'s work (Hou et al. 2020)	Matsui's Algorithm 1
3-round	t [1]	22	24	26	-	-
	$N \times t$ [2]	22	24	26	32	26
	Success Rate [3]	97.17%	97.79%	98.06%	95%	97.99%
4-round	t	20	18	22	-	-
	$N \times t$	260	252	264	332	269
	Success Rate	97.12%	97.61%	97.79%	95%	97.89%
5-round	t	130	70	140	-	-
	$N \times t$	2730	2730	2660	8631	2751
	Success Rate	97.61%	97.61%	97.64%	90%	97.68%
6-round	t	120	34	48	-	-
	$N \times t$	68880	68918	68928	-	68939
	Success Rate	97.06%	97.33%	97.77%	-	97.63%

[1]The length of single training sample. [2]The size of plaintext-ciphertext pairs set. [3]The success rate is quoted for 10^4 trials.

Computational Complexity. The running time focus on the GPU time. And for the existing GPU devices (i.e. Nvidia GeForce RTX3090), the time is negligible.

Comparison with previous works

In order to allow for a direct comparison to existing literature, we also perform the traditional key-recovery attacks using **Matsui's Algorithm 1**. And for **Matsui's Algorithm 1**, we focus on the success rate under using the same number of plaintexts. For Hou et al.'s methods and Zhou et al.'s methods, we focus on the numbers of rounds and the success rates.

Comparison with Matsui's algorithm 1. In Matsui (1993), Matsui proposes the linear cryptanalysis using linear approximate expressions. And we perform traditional one bit key-recovery attacks on 3-, 4-, 5- 6-round DES using **Matsui's algorithm 1**. For comparison, the number of the plaintexts used in **Matsui's algorithm 1** is $\left({p_r}^i - 0.5\right)^{-2}$, which is the same number of plaintexts as ML-aided one bit key-recovery attacks. The success rate is 97.99%, 97.89%, 97.68%, 97.63% for 3-, 4-, 5-, 6-round DES, respectively. As shown in Table 5, the success rate of using the ML-aided method is closed to using **Matsui's algorithm 1**, which indicates that ML-aided cryptanalysis can achieve the same performance as orthodox counterparts. The result shows that ML-aided one bit key-recovery attack can be a useful addition for the security of symmetric ciphers.

Comparison with Hou *et al.*'s methods. In Hou et al. (2020), Hou et al. design linear attacks using deep learning and apply to round-reduced DES. Using Hou et al.'s methods, it requires 32, 332, 8631 plaintexts to achieve the success rate reaching 95% for 3-, 4- and 90% for 5-round DES (see the penultimate column of Table 5). In other words, Hou et al.'s methods need more plaintext than the ML-aided one bit key-recovery attack, which indicates that our method has considerable advantages in success rate and data complexity over attacks devised using Hou et al.'s methods.

Comparison with Zhou et al.'s methods. In Zhou et al. (2023), Zhou et al. train new neural-linear distinguishers and perform key-recovery attacks for round-reduced DES. And Zhou et al.'s work presents better key-recovery attacks than Hou et al.'s work. But Zhou et al.'s methods don't also reach the baseline of traditional counterparts. Compared by Zhou et al.'s work, our distinguishers perform the better key-recovery attacks given by our distinguishers, which is shown in Table 6.

Table 6 Comparison with Zhou et al. (2023) on the success rate of one bit key-recovery attacks

Target		ML-aided one bit key-recovery attack	Zhou et al. (2023)
3-round	t [1]	32	32
	$N \times t$ [2]	32	32
	Success Rate [3]	98.96%	98.5%
4-round	t	18	256
	$N \times t$	252	256
	Success Rate	97.61%	96.6%
5-round	t	28	1024
	$N \times t$	1008	1024
	Success Rate	88.39%	84.28%
6-round	t	42	1024
	$N \times t$	2016	2048
	Success Rate	63.64%	60.6%

[1]The length of single training sample. [2]The size of plaintext-ciphertext pairs set. [3]The success rate is quoted for 10^4 trials.

ML-aided multiple bits key recovery

Attack architecture

In Matsui (1993), Matsui proposes **Matsui's algorithm 2** to utilize r-round linear approximate expressions to attack $(r+x)$-round target ciphers $(x > 1)$. As shown in Fig. 1, for the r-round linear approximate expression, if the guess of K_0 and K_{r+1} is correct, the linear expressions $\alpha \cdot P' \oplus \beta \cdot C' \oplus \gamma \cdot K' = 0$ holds with probability $p_r \neq 0.5$. If the guess of K_0 and K_{r+1} is incorrect, the effectiveness of this equation clearly decreases and the probability is close to 0.5.

Generation of the Training Data. Considering that $\alpha \cdot P' \oplus \beta \cdot C' \oplus \gamma \cdot K'$ obeys the different Bernoulli distributions related to whether the guessed keys are correct or not, we design a new training data format to train the neural networks distinguishing the different Bernoulli distributions, which is shown in Eq. 7. In Eq. 7, $\omega_i = \alpha \cdot P_i \oplus \beta \cdot C_i \oplus \gamma \cdot K, i \in [1, t]$.

$$Y(x_1||x_2||...||x_t) = \begin{cases} 1 \ if \ x_i = \omega_i \ (x_i \sim Bern(1-p_r)) \\ 0 \ if \ x_i \sim Bern\left(\frac{1}{2}\right) \end{cases} \tag{7}$$

In Eq. 7, one positive sample uses t plaintexts $\{P_1, P_2, ..., P_t\}$ and their corresponding ciphertexts $\{C_1, C_2, ..., C_t\}$, where these plaintexts are encrypted r rounds to these ciphertexts using one master key. In other words, there are N master keys used in N positive samples.

Training Schemes. Similar to the one bit key-recovery attacks, the neural networks used in multiple bits key-recovery attacks are the same as the neural networks

used in Gohr (2019). And in order to be consistent with previous works of neural cryptanalysis, the trained neural networks are called r-round neural distinguishers if the training data is generated using r-round linear approximate expressions.

Key-recovery attack. Given the candidate key K_{guess}, use K_{guess} to encrypt P to P' and decrypt C to C'. If K_{guess} is incorrect, $\alpha \cdot P' \oplus \beta \cdot C'$ obeys the random distribution. And in one key-recovery attack, the value of $\gamma \cdot K'$ is fixed. So whether the guess of $\gamma \cdot K'$ is correct or not, $\alpha \cdot P' \oplus \beta \cdot C' \oplus \gamma \cdot K'$ obeys the random distribution, which makes the trained neural network outputs a value less than 0.5. If K_{guess} is correct, guessing the value of $\gamma \cdot K'$, the trained neural network outputs the higher value using correct guess than using incorrect guess. Algorithm 2 gives details on multiple bits key-recovery attack. Similar to ML-aided one bit key-recovery attack, there are three main parameters need to be fixed in Algorithm 2, which are the linear approximate expression L_r, the length of single training sample t and the size of plaintext-ciphertext pairs set $N \times t$, respectively. These three parameters are related to the success rate.

Algorithm 2 ML-aided multiple bits key-recovery attack

Require: r-round linear approximate expression L_r: $\alpha \cdot P' \oplus \beta \cdot C' = \gamma \cdot K'$
Trained Neural Network: NN_t^r
Plaintext-Ciphertext pairs set: $S = \{(P_1, C_1), ..., (P_{N\times t}, C_{N\times t})\}$
Candidate Key set: $GK = \{GK_1, GK_2, ..., GK_l\}$
Ensure: $\gamma \cdot K'$, RK

$score \leftarrow -\infty$
$RK \leftarrow 0$
$\gamma \cdot K' \leftarrow 0$
for $gk \in \{GK_1, GK_2, ..., GK_l\}$ **do**
  $P'_i \leftarrow Encrypt_{gk}(P_i)$ for $i \in \{1, ..., N \times t\}$
  $C'_i \leftarrow Decrypt_{gk}(C_i)$ for $i \in \{1, ..., N \times t\}$
  $v_i^0 \leftarrow NN_t^r\left(\alpha \cdot P'_{t\times i+1} \oplus \beta \cdot C'_{t\times i+1} || ... || \alpha \cdot P'_{t\times i+t} \oplus \beta \cdot C'_{t\times i+t}\right)$ for $i \in \{0, ..., N-1\}$
  $w_0 \leftarrow \sum_{i=0}^{N-1} \log_2\left(\frac{v_i^0}{1-v_i^0}\right)/N$
  $v_i^1 \leftarrow NN_t^r\left(\alpha \cdot P'_{t\times i+1} \oplus \beta \cdot C'_{t\times i+1} \oplus 1 || ... || \alpha \cdot P'_{t\times i+t} \oplus \beta \cdot C'_{t\times i+t} \oplus 1\right)$ for $i \in \{0, ..., N-1\}$
  $w_1 \leftarrow \sum_{i=0}^{N-1} \log_2\left(\frac{v_i^1}{1-v_i^1}\right)/N$
  if $w_0 > w_1$ **then**
    if $w_0 > score$ **then**
      $\gamma \cdot K' \leftarrow 0$
      $RK \leftarrow gk$
      $score \leftarrow w_0$
    end if
  else
    if $w_1 > score$ **then**
      $\gamma \cdot K' \leftarrow 1$
      $RK \leftarrow gk$
      $score \leftarrow w_1$
    end if
  end if
end for
Return $\gamma \cdot K'$, RK

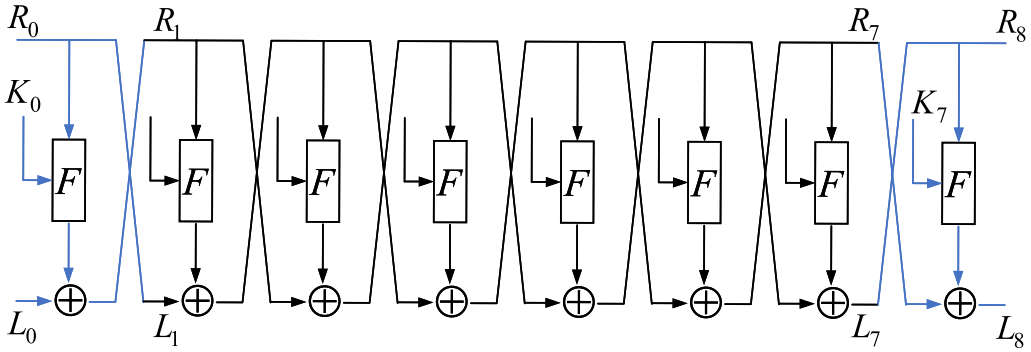


Fig. 5 Multiple bits attack of $(1+6+1)$-round Des

Application to Des

In this section, we perform a key-recovery attack on 8-round Des, which covers more rounds than Hou et al. (2020).

Multiple bits key-recovery attack of 8-round Des . Before performing the key-recovery attack of 8-round Des, we train the 6-round neural distinguisher using the 6-round linear approximate expression L_6 shown in Table 4. And the length of a single sample is t. As shown in Fig. 5, we guess K_0 and K_7 to obtain $L_1||R_1$ and $L_7||R_7$, and perform key-recovery attacks using 6-round neural distinguishers. Considering that only $L_1[7,18,24]$, $L_7[7,18,24,29]$ and $R_7[15]$ are used in 6-round neural distinguishers, we just need to guess $K_0[18,19,20,21,22,23]$ and $K_7[42,43,44,45,46,47]$ in Algorithm 2. And the one bit in Algorithm 2 is $K_2[22] \oplus K_3[44] \oplus K_4[22] \oplus K_6[22]$. We perform multiple key-recovery attacks using different parameters $(N \times t = 4 \times 10^5, t = 80)$, $(N \times t = 4.8 \times 10^5, t = 80)$, $(N \times t = 5.6 \times 10^5, t = 80)$, $(N \times t = 6.4 \times 10^5, t = 80)$ and their success rates, with quoting for 10^3 trials, are 80.2%, 88.4%, 93.6% and 96.5%, respectively.

Data Complexity. There are $N \times t$ plaintexts used in one key-recovery attack.

Computational Complexity. The running time is related to the number of candidate keys and the number of plaintexts. In this attack of 8-round Des, there are 2^{12} candidate keys ($K_0[18,19,20,21,22,23]$, $K_7[42,43,44,45,46,47]$) in total. And the time complexity is $\frac{2^{12} \times N \times t}{8}$ executions of 8-round Des.

Comparison with previous work

In this section, we also perform the key-recovery attacks using **Matsui's algorithm 2** and previous machine learning-aided multiple bits key-recovery attacks.

Comparison with Matsui's algorithm 2. We perform the traditional key-recovery attack of 8-round Des using the 6-round linear approximate expression L_6 (see Table 4). And the success rate is 78.6% using 4×10^5 plaintexts, which is lower than the success rate of ML-aided multiple bits key-recovery attack. The result shows that machine learning can do cryptanalysis at a level that is interesting for a cryptographer and present an effective cryptographic evaluators' tool box.

Comparison with Hou *et al.*'s methods. In Hou et al. (2020), Hou et al. show the results of 4-round key-recovery attacks by using their 3-round neural distinguisher. As a comparison, based on our neural distinguisher of 3-round Des, the success rate using Algorithm 2 is 99.86% with using $(N \times t = 256, t = 16)$ as the parameters. Hou et al.'s success rate is far lower than the success rate of using out method. Compared to the results in Hou et al. (2020), for attacking 4-round, the success rate of Algorithm 2 improves considerably using less data. Most importantly, more rounds are covered in our work.

Comparison with Zhou *et al.*'s methods. In Zhou et al. (2023), Zhou et al.'s multiple bits key-recovery attacks surpasses Hou et al.'s results. For comparison, we perform the multiple bits key-recovery attacks using they same (or less) number of plaintexts as Zhou et al.'s work. The key recovery attacks of 4-round Des (resp. 5-round Des) are performed using the 3-round neural-linear distinguisher (resp. the 4-round neural-linear distinguisher). With $(N \times t = 32, t = 16)$, the mean rank of the real key is no more than 8 in 10^3 trials for the 4-round attack using our distinguishers and algorithm. And the mean of rank of the real key is no more than 13 in 10^3 trials for the 5-round attack under $(N \times t = 240, t = 20)$. As a comparison, the mean of key rank is around 30 in Zhou et al.'s work. The results of multiple bits key recovery show that our method has considerable advantages in key rank.

Conclusions

This paper mainly focuses on machine learning aided linear cryptanalysis. By carefully studying how to distinguish different Bernoulli distributions using neural networks, we build the relationship between neural networks and approximation expressions. Based on this, we design a valid machine learning-aided linear cryptanalysis framework and apply it to round-reduced DES. The results in this paper surpass the previous machine learning-aided linear cryptanalysis, which provides a new benchmark in machine learning-aided linear cryptanalysis. And the result of linear cryptanalysis shows that machine learning aided cryptanalysis can achieve the same performance as classical methods.

Our work indicates that machine learning can perform well not only in differential cryptanalysis, but also in linear cryptanalysis. Our model focuses primarily on the probability of approximation expressions. Like differential-neural cryptanalysis, constructing machine learning aided linear cryptanalysis that neural networks can use multiple output masks under one input mask is left as future work. And another improved work can be done from the key-recovery strategy. Designing a better key-recovery strategy matching neural distinguishers will be effective to reduce the time complexity.

Appendix 1: Neural-linear distinguishers using different neural networks

Considering that several state-of-the-art neural network structures have been developed, more and more researchers choose appropriate neural network structures to training neural distinguishers. In Bao et al. (2022), Bao et al. use the Residual Neural Network (ResNet) (He et al. 2016), Dense Neural Network (DenseNet) (Huang et al. 2017) and Squeeze-and-Excitation Neural Network (SENet) (Hu et al. 2018) to training neural-differential distinguishers for round-reduced SIMON32/64. And it is found that SENet yields distinguishers that are superior to that of the other two. So it is important to try different network structures and choose the best distinguisher.

In light of this, we train the neural-linear distinguishers using different neural networks for 4-, 5-, 6-round DES. The accuracy using different neural networks is shown in Table 7. As shown in Table 7, it has no obvious gap in the accuracy using different neural networks. Considering of the higher cost of time in training distinguishers using SENet and DenseNet, we choose the ResNet to train neural distinguishers in our research.

Table 7 Summary of neural-linear distinguishers using different neural networks for one bit key-recovery attack of DES

Target Cipher	Length of single sample	Network	Accuracy*
4—round DES	32	ResNet	75.38%
		SENet	75.41%
		DenseNet	75.31%
	64	ResNet	83.48%
		SENet	83.58%
		DenseNet	83.54%
	128	ResNet	91.29%
		SENet	91.31%
		DenseNet	91.70%
5—round DES	128	ResNet	66.61%
		SENet	66.15%
		DenseNet	66.72%
	256	ResNet	72.92%
		SENet	72.93%
		DenseNet	72.96%
	512	ResNet	80.43%
		SENet	80.56%
		DenseNet	80.47%
6—round DES	256	ResNet	54.75%
		SENet	54.80%
		DenseNet	54.94%
	512	ResNet	56.67%
		SENet	56.83%
		DenseNet	56.83%
	768	ResNet	58.30%
		SENet	58.22%
		DenseNet	58.15%

*The accuracy is quoted for 2^{20} samples.

Table 8 Comparison of one bit key-recovery attack

Target Cipher	Number of plaintexts	Success rate of Zhou et al. (2023)[1]	Success rate of Matsui's Algorithm 1[2]
3-round	32	98.5%	98.842%
4-round	256	96.6%	97.451%
	512	99.5%	99.737%
5-round	1024	84.28%	88.967%
	2048	91.15%	95.532%
	4617	-	99.417%
6-round	2048	60.6%	63.465%

[1]The results are from Zhou et al. (2023). [2]The success rate is quoted for 10^5 trials.

Appendix 2: Comparison in Zhou et al. (2023) and Matsui's Algorithm 1

The success rate (90% using 4617 plaintexts) of the traditional method given by Zhou et al. (2023) is wrong. In Matsui (1994), Matsui gives a lemma to describe the success rate of **Matsui's Algorithm 1**. And the success rate is about 92.1% using $0.5 \times |p - 0.5|^{-2}$ plaintexts, where the p is the probability of the linear approximate expression. Considering of that the probability of the 5-round expression used inZhou et al. (2023) is $0.5 + 1.22 \times 2^{-6}$, the success rate of **Matsui's Algorithm 1** is more than 90% only using about $0.5 \times \left|1.22 \times 2^{-6}\right|^{-2} \approx 1376$ plaintexts (far below 4617). And the success rate of **Matsui's Algorithm 1** is 99.417% using 4617 plaintexts for 10^5 trials.

We recalculate the success rate of **Matsui's Algorithm 1** by performing a lot of experiments. The correct success rate of **Matsui's Algorithm 1** is shown in Table 8.

Acknowledgements
The authors would like to thank the reviewers for their constructive comments that have greatly improved the paper.

Author contributions
Zezhou Hou designed and performed experiments, analysed data and wrote the paper. Jiongjiong Ren designed the experiments and prepared the manuscript. Shaozhen Chen gave technical support and conceptual advice. All authors discussed the results and implications and commented on the manuscript at all stages.

Funding
This work is supported by the National Natural Science Foundation of China (No. 62206312).

Data availability
Data sharing is not applicable to this article as no datasets were generated or analysed during the current study. Supplementary code and data for this paper is available at https://github.com/ml-aided-crypto/machine-learning-aided-linear-cryptanalysis.git.

Declarations

Competing interests
The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Received: 7 July 2024 Accepted: 10 October 2024
Published online: 01 April 2025

References
Abadi M, Andersen DG (2016) Learning to protect communications with adversarial neural cryptography. Preprintat at. https://arxiv.org/abs/1610.06918

Bao Z, Guo J, Liu M, Ma L, Tu Y (2022) Enhancing differential-neural cryptanalysis. In: Agrawal S, Lin D (eds) Advances in cryptology - ASIACRYPT 2022–28th International conference on the theory and application of cryptology and information security. Springer, Heidelberg, pp 318–347. https://doi.org/10.1007/978-3-031-22963-3_11

Bao Z, Guo J, Liu M, Ma L, Tu Y (2021) Enhancing differential-neural cryptanalysis. IACR Cryptol. ePrint Arch. Preprint at https://eprint.iacr.org/2021/719

Beaulieu R, Shors D, Smith J, Treatman-Clark S, Weeks B, Wingers L (2013) The SIMON and SPECK families of lightweight block ciphers. IACR Cryptol. ePrint Arch. Preprint at http://eprint.iacr.org/2013/404

Benamira A, Gérault D, Peyrin T, Tan QQ (2021) A deeper look at machine learning-based cryptanalysis. In: Canteaut A, Standaert F (eds) Advances in cryptology - EUROCRYPT 2021–40th annual international conference on the theory and applications of cryptographic techniques. Springer, Heidelberg, pp 805–835. https://doi.org/10.1007/978-3-030-77870-5_28

Biham E, Chen R (2004) Near-collisions of SHA-0. In: Franklin MK (ed) Advances in cryptology - CRYPTO 2004, 24th annual international cryptology conference, vol 3152. Springer, Heidelberg, pp 290–305. https://doi.org/10.1007/978-3-540-28628-8_18

Courtois NT (2004) Feistel schemes and bi-linear cryptanalysis. In: Franklin MK (ed) Advances in cryptology - CRYPTO 2004, 24th annual international cryptology conference. Springer, Heidelberg, pp 23–40. https://doi.org/10.1007/978-3-540-28628-8_

Dong T, Huang T (2020) Neural cryptography based on complex-valued neural network. IEEE Trans Neural Networks Learn Syst 31(11):4999–5004. https://doi.org/10.1109/TNNLS.2019.2955165

Gohr A (2019) Improving attacks on round-reduced speck32/64 using deep learning. In: Boldyreva A, Micciancio D (eds) Advances in cryptology - CRYPTO 2019–39th annual international cryptology conference. Springer, Heidelberg, pp 150–179. https://doi.org/10.1007/978-3-030-26951-7_6

Harpes C, Kramer G, Massey JL (1995) A generalization of linear cryptanalysis and the applicability of matsui's piling-up lemma. In: Guillou LC, Quisquater J (eds) Advances in cryptology - EUROCRYPT '95, international conference on the theory and application of cryptographic techniques. Springer, Heidelberg, pp 24–38. https://doi.org/10.1007/3-540-49264-X_3

He K, Zhang X, Ren S, Sun J (2016) Deep residual learning for image recognition. 2016 IEEE Conference on computer vision and pattern recognition, CVPR 2016. IEEE Computer Society, Las Vegas, pp 770–778. https://doi.org/10.1109/CVPR.2016.90

Hermelin M, Cho JY, Nyberg K (2008) Multidimensional linear cryptanalysis of reduced round serpent. In: Mu Y, Susilo W, Seberry J (eds) Information security and privacy, 13th Australasian Conference, ACISP 2008. Springer, Heidelberg, pp 203–215. https://doi.org/10.1007/978-3-540-70500-0_15

Hou B, Li Y, Zhao H, Wu B (2020) Linear attack on round-reduced DES using deep learning. In: Chen L, Li N, Liang K, Schneider SA (eds) Computer security - ESORICS 2020–25th European symposium on research in computer security. Springer, Cham, pp 131–145. https://doi.org/10.1007/978-3-030-59013-0_7

Hu J, Shen L, Sun G (2018) Squeeze-and-excitation networks. 2018 IEEE conference on computer vision and pattern recognition, CVPR 2018. IEEE Computer Society, Salt Lake City, pp 7132–7141. https://doi.org/10.1109/CVPR.2018.00745

Huang G, Liu Z, Maaten L, Weinberger KQ (2017) Densely connected convolutional networks. 2017 IEEE conference on computer vision and pattern recognition, CVPR 2017. IEEE Computer Society, Honolulu, pp 2261–2269. https://doi.org/10.1109/CVPR.2017.243

Kaliski Jr BS, Robshaw MJB (1994) Linear cryptanalysis using multiple approximations. In: Desmedt Y (ed) Advances in cryptology - CRYPTO '94, 14th annual international cryptology conference. Springer, Heidelberg, pp 26–39. https://doi.org/10.1007/3-540-48658-5_4

Kim J, Picek S, Heuser A, Bhasin S, Hanjalic A (2019) Make some noise. unleashing the power of convolutional neural networks for profiled side-channel analysis. IACR Trans Cryptogr Hardw Embed Syst. https://doi.org/10.13154/tches.v2019.i3.148-179

Knudsen LR, Robshaw MJB (1996) Non-linear approximations in linear cryptanalysis. In: Maurer UM (ed) Advances in cryptology - EUROCRYPT '96, international conference on the theory and application of cryptographic techniques. Springer, Heidelberg, pp 224–236. https://doi.org/10.1007/3-540-68339-9_20

Langford SK, Hellman ME (1994) Differential-linear cryptanalysis. In: Desmedt Y (ed) Advances in cryptology - CRYPTO '94, 14th annual international cryptology conference. Springer, Heidelberg, pp 17–25. https://doi.org/10.1007/3-540-48658-5_3

Li Y, Deng S, Xiao D (2011) A novel hash algorithm construction based on chaotic neural network. Neural Comput Appl. https://doi.org/10.1007/s00521-010-0432-2

Maghrebi H, Portigliatti T, Prouff E (2016) Breaking cryptographic implementations using deep learning techniques. In: Carlet C, Hasan MA, Saraswat

V (eds) Security, privacy, and applied cryptography engineering - 6th international conference, SPACE 2016. Springer, Heidelberg, pp 3–26
Matsui M (1994) The first experimental cryptanalysis of the data encryption standard. In: Desmedt Y (ed) Advances in cryptology - CRYPTO '94, 14th annual international cryptology conference. Springer, Heidelberg, pp 1–11
Matsui M (1993) Linear cryptanalysis method for DES cipher. In: Helleseth T (ed) Advances in cryptology - EUROCRYPT '93, workshop on the theory and application of of cryptographic techniques. Springer, Heidelberg, pp 386–397. https://doi.org/10.1007/3-540-48285-7_33
Rivest RL (1991) Cryptography and machine learning. In: Imai H, Rivest RL, Matsumoto T (eds) Advances in cryptology - ASIACRYPT '91, international conference on the theory and applications of cryptology. Springer, Heidelberg, pp 427–439
Standards NB (1977) Data encryption standard (DES)
Zhou R, Duan M, Wang Q, Wu Q, Guo S, Guo L, Gong Z (2023) Neural-linear attack based on distribution data and its application on DES. In: Lin, H. (ed.) International Workshop on Computer Science and Engineering - WCSE 2023, pp. 64-73 . https://10.18178/wcse.2023.06.011

Publisher's Note